Security Engineering
Flask Tutorial 2

Srđan Krstić Daniel Galán Hoàng Nguyễn
Institute of Information Security
Department of Computer Science
ETH Zürich

Contents

1 Introduction

In the previous tutorial, we explored Flask's fundamental features, including sessions and templates, and applied them to build the `Thoughts` web application.

This tutorial takes a two-part approach. First, we examine a simplified example to learn database persistence using SQLAlchemy. Then, we return to the `Thoughts` application to implement more advanced features: data persistence, user authentication, and security through manual access control policies.

The current version of the `Thoughts` application requires user registration every time the web server restarts because it does not *persistently* save its state. We will improve the implementation by adding a simple database and showing how a Flask application can interact with it. In fact, any production-ready application necessarily requires persistent storage.

The current implementation also handles user authentication in an ad hoc manner. We will improve this using another Flask library.

Finally, we will enhance the security of the Thoughts application by enforcing some access control policies.

2 Flask-SQLAlchemy Library Overview

SQLAlchemy [1] is a Python library and an object relational mapper (ORM) that gives developers a convenient API to perform queries over a database. An ORM translates a class structure of an object-oriented program into a relational schema. It also translates objects (class instances) into schema-conformant relations, which are then stored as database tables. The translation is reversible: it preserves the objects' structure and their relationships so that they can be read from the tables and instantiated as objects when needed. Objects translated with an ORM are said to be persistent.

Our starting point is the `minisql-template`, which can be downloaded from Moodle, under the second week. In this section, we interactively add code to this template. As with all exercises, you never need to copy code from PDFs: we also provide a second template `minisql` template with all changes incorporated.

Installation SQLAlchemy comes pre-installed in the provided Docker template. If you are working in a local environment instead, SQLAlchemy is included as a dependency in the `requirements.txt` file and will be installed automatically when you set up the project.

For detailed local setup instructions, refer to the setup guide provided in the first week of the course.

Execution Unlike other Flask-based templates, we will run this Docker container with a terminal interface where you can run commands like `python` or `sqlite3`, rather than automatically launching Flask. To achieve this, use the command:

```
docker-compose run --rm flask bash
```

[1] `https://docs.sqlalchemy.org/en/20/`

Data model Let's start using Flask-SQLAlchemy by creating a simple data model. Consider the following code:

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'

db = SQLAlchemy(app)

class Person(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50))

    def __repr__(self):
        return f'<Person id:{self.id} name:{self.name}>'
```

We provide example code in the `sql.py` file. After creating a Flask application instance `app`, we use it to create an SQLAlchemy object `db`. Then we create a class `Person` with a unique (integer) id and a (string) name. All classes that inherit from the `db.Model` class are part of the data model and are used by the ORM to construct a relational schema. For simplicity, we configure the `app` instance to use SQLite. Other kinds of databases (e.g., MySQL, Oracle, Postgres) are also supported.

We will now create the actual database in the filesystem. This is achieved by running `db.create_all()`. Either call this in your script or start a Python interpreter and call the function explicitly:

```
python3
>>> from sql import db, app
>>> app.app_context().__enter__()
>>> db.create_all()
```

This creates an SQLite database file (`instance/app.db`) and initializes its schema to match the models defined in the Python code

You can inspect the database as follows: [2]

```
sqlite3 instance/app.db
sqlite> .tables
person
sqlite> .schema
CREATE TABLE person (
        id INTEGER NOT NULL,
        name VARCHAR(50),
        PRIMARY KEY (id)
);
```

[2]You can also use DB Browser for SQLite `https://sqlitebrowser.org/` to inspect your database.

```
sqlite> .quit
```

As you can see the `Person` class is declared as a `person` table in the database. Note that its name is not capitalized.

Add, update, and delete To query the state of the database, use the `query` property of each class in the data model. To change the state of the database, use the `session` property of the `db` object:

```
python3
>>> from sql import db, app, Person
>>> app.app_context().__enter__()
>>> Person.query.all()
[]
>>> alice = Person(name='Alice')
>>> db.session.add(alice)
>>> db.session.commit()
>>> Person.query.all()
[<Person id:1 name:Alice>]
```

At the moment table `person` in the database has one row for Alice. When querying each row is used to instantiate an object of the appropriate data model class. Here we query a list with a single `Person` object.

We can also update existing rows in the table by updating object fields:

```
>>> alice.name='Alicia'
>>> db.session.commit()
>>> Person.query.all()
[<Person id:1 name:Alicia>]
```

To delete a row, pass the corresponding object to `session.delete`:

```
>>> bob = Person(name='Bob')
>>> db.session.add(bob)
>>> db.session.commit()
>>> Person.query.all()
[<Person id:1 name:Alicia>, <Person id:2 name:Bob>]
>>> db.session.delete(alice)
>>> db.session.commit()
>>> Person.query.all()
[<Person id:2 name:Bob>]
```

If we add a new person named Bob to the database and then delete the `alice` object, only Bob remains.

Basic queries The `Person.query` method resembles a simple `SELECT` SQL query with a `FROM` clause fixed to the table `person`. Suppose we add Alice back to the database, here are some basic queries we can do:

```
>>> Person.query.all()
[<Person id:2 name:Bob>, <Person id:3 name:Alice>]
>>> Person.query.filter_by(name='Bob').all()
[<Person id:2 name:Bob>]
Person.query.filter(Person.name.startswith('A')).all()
[<Person id:3 name:Alice>]
>>> Person.query.count()
2
>>> Person.query.order_by(Person.name.desc()).all()
[<Person id:2 name:Bob>, <Person id:3 name:Alice>]
>>> Person.query.get(2)
<Person id:2 name:Bob>
```

The `WHERE` clause is created using the `filter_by(...)` or `filter(...)` methods. The former simply filters the object based on one of their fields, while the latter can evaluate a custom predicate on the object. Simple aggregation operators (e.g., `count()`) and the result can be ordered, which corresponds to the `ORDER BY` SQL clause.

Finally, we can use the `get(id)` method to obtain an object based on its primary key value `id`.

Chaining these methods only builds the SQL query – it does not yet evaluate it on the database. Only once you call the (`all()` or) `first()` method is the built SQL query evaluated and you a get the actual object(s).

Associations Let's first see how to build associations with one-to-many multiplicity. We start by adding another data model class:

```
class Thought(db.Model):
  id = db.Column(db.Integer, primary_key=True)
  content = db.Column(db.String(500))
  def __repr__(self):
    end = '...\'>' if len(self.content) > 10 else '\'>'
    return f'<Thought id:{self.id} content:\'{self.content[:10]}' + end
```

To add a created-createdBy one-to-many association between `Person` and `Thought` classes, we add an association end to each class:

```
class Person(db.Model):
    #...
    created = db.relationship('Thought', backref='createdBy')

class Thought(db.Model):
    #...
    person_id = db.Column(db.Integer, db.ForeignKey('person.id'))
```

The first additional line uses a special `db.relationship` method, which takes the string *name* of the opposite side's class as parameter. The new field is considered a collection of `Though` objects. Adding the `backref='createdBy'` parameter creates a field `createdBy` of type `Person` in the `Thought` class that points to the `Person` object on the reverse side of the association. In the `Thought` class we need to add an integer id field, which has a foreign key constraint to the `id` column in the table `person` (hence the non-capitalized name). We can now recreate the database and inspect the schema:

```
python3
>>> from sql import db
>>> db.create_all()
sqlite3 app.db
sqlite> .schema
CREATE TABLE person (
        id INTEGER NOT NULL,
        name VARCHAR(50),
        PRIMARY KEY (id)
);
CREATE TABLE thought (
        id INTEGER NOT NULL,
        content VARCHAR(500),
        person_id INTEGER,
        PRIMARY KEY (id),
        FOREIGN KEY(person_id) REFERENCES person (id)
);
```

We see that at the level of database tables one-to-many association is implemented as an extra column in the `thought` table with a foreign key constraint to the primary key of the `person` table. Let's see how SQLAlchemy uses these keys to populate the fields of the objects:

```
python3
>>> from sql import db, Person, Thought
>>> alice = Person(name='Alice')
>>> bob = Person(name='Bob')
>>> db.session.add_all([alice, bob])
>>> db.session.commit()
>>> t1 = Thought(content='I think, therefore I am.', createdBy=alice)
>>> t2 = Thought(content='Cogito, ergo sum',createdBy=bob)
>>> db.session.add_all([t1,t2])
>>> db.session.commit()
>>> alice.created
[<Thought id:1 content:'I think, t...'>]
>>> bob.created
[<Thought id:2 content:'Cogito, er...'>]
>>> t1.createdBy
<Person id:1 name:Alice>
>>> t2.createdBy
<Person id:2 name:Bob>
```

As you can see the `created` and `createdBy` fields point to the right objects (not to ids). This is maintained even if the objects are fetched from the database:

```
>>> del alice
>>> del bob
>>> alice = Person.query.get(1)
>>> bob = Person.query.get(2)
>>> alice.created
[<Thought id:1 content:'I think, t...'>]
>>> bob.created
[<Thought id:2 content:'Cogito, er...'>]
```

Now let's create a many-to-many association to encode that people can vote for thoughts. This is done by manually creating another table and adjusting one of the two classes in the association:

```
votes = db.Table('votes',
  db.Column('person_id', db.Integer, db.ForeignKey('person.id')),
  db.Column('thought_id', db.Integer, db.ForeignKey('thought.id'))
)

class Person(db.Model):
  #...
  voted = db.relationship('Thought', secondary=votes, backref='votedBy')
```

The `votes` table has two columns: each an id with foreign key constraint to one of the two tables encoding the associated classes. The additional `voted` field in the `Person` class is sufficient to tell the ORM to create and maintain collection fields `voted` and `votedBy` in the `Person` and `Thought` classes. If we inspect the new schema, we have the additional table as expected:

```
sqlite> .schema
CREATE TABLE person (
        id INTEGER NOT NULL,
        name VARCHAR(50),
        PRIMARY KEY (id)
);
CREATE TABLE thought (
        id INTEGER NOT NULL,
        content VARCHAR(500),
        person_id INTEGER,
        PRIMARY KEY (id),
        FOREIGN KEY(person_id) REFERENCES person (id)
);
CREATE TABLE votes (
        person_id INTEGER,
        thought_id INTEGER,
        FOREIGN KEY(person_id) REFERENCES person (id),
        FOREIGN KEY(thought_id) REFERENCES thought (id)
);
```

This association can be used as follows:

```
>>> alice.voted.append(t1)
>>> alice.voted.append(t1)
>>> alice.voted
```

```
[<Thought id:1 content:'I think, t...'>, <Thought id:1 content:'I
    ↪ think, t...'>]
>>> db.session.commit()
>>> alice.voted
[<Thought id:1 content:'I think, t...'>]
```

As you can see the associations have set semantics in SQLAlchemy. Even though the underlying `votes` table has two identical rows, SQLAlchemy's ORM only uses them to establish a relationship between the objects. Multiplicity can still be queried, but it requires using a low-level `query(...)` method of the `db.session` object, which directly returns a list of table rows (as tuples).

```
>>> ids = db.session.query(Thought.id)
                    .filter((votes.c.person_id==alice.id) &
                            (votes.c.thought_id==Thought.id))
                    .all()
>>> [Thought.query.get(i) for i in ids]
[<Thought id:1 content:'I think, t...'>, <Thought id:1 content:'I
    ↪ think, t...'>]
```

To use the ORM to obtain multiple votes requires redesigning the data model. Let's delete the `voted` table and introduce `Vote` data model class that has one-to-many associations with both `Person` and `Thought` classes:

```
class Vote(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    person_id = db.Column(db.Integer, db.ForeignKey('person.id'))
    thought_id = db.Column(db.Integer, db.ForeignKey('thought.id'))

class Person(db.Model):
    # ...
    voted = db.relationship('Vote', backref='voter')

class Thought(db.Model):
    # ...
    votedBy = db.relationship('Vote', backref='votee')
```

Now we can use the ORM directly:

```
>>> v1 = Vote(voter=alice, votee=t1)
>>> v2 = Vote(voter=alice, votee=t1)
>>> db.session.add_all([v1,v2])
>>> db.session.commit()
>>> [x.votee for x in alice.voted]
[<Thought id:1 content:'I think, t...'>, <Thought id:1 content:'I
    ↪ think, t...'>]
```

The last line produces a multiset of Thoughts.

To see the complete implementation with all changes from this section, download the `minisql` template from Moodle. This template contains the template `minisql-template` with all code modifications applied.

2.1 Persistence in the Thoughts Application

We begin with last week's task solution as our starting template for a new iteration of the `Thoughts` app. You can use your own implementation or download the provided template `thoughts-simple` from the previous week's section on Moodle. The tutorial explanations are based on our provided template.

We start by importing, configuring and instantiating the `db` instance of `SQLAlchemy`. We then replace the non-persistent data model with its persistent analogous:

```
class Vote(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    person_id = db.Column(db.Integer, db.ForeignKey('person.id'))
    thought_id = db.Column(db.Integer, db.ForeignKey('thought.id'))

class Person(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50), unique=True)
    password = db.Column(db.String(50))
    created = db.relationship('Thought', backref='createdBy')
    voted = db.relationship('Vote', backref='voter')

class Thought(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    content = db.Column(db.String(500))
    person_id = db.Column(db.Integer, db.ForeignKey('person.id'))
    votedBy = db.relationship('Vote', backref='votee')
```

Clearly, the implementation of the routes needs to be adjusted: we cannot no longer refer directly to the lists of Person and Thought objects. For example, in the `index` route, we cannot assign the (now undefined) `thoughts` list to the `table` variable when rendering the `index.html` template. Instead, we must use `Thought.query.all()` to retrieve the thoughts from the database.

```
@app.route('/')
def index():
+   thoughts=Thought.query.all()
    return render_template('index.html', table=thoughts)
```

Task 1: Persistence on `Thoughts`

Retrieve last week's task implementation to build upon it. Use your own work or download the provided `thoughts-simple` template from last week's section on Moodle.

. . . .

Implement the remaining routes for the `Thoughts` application using SQLAlchemy for data persistence as described above.
Save your completed work, as it will serve as the starting point for the next task in this tutorial.

. . . .

After the corresponding deadline, the solution will be published in Moodle as `thoughts-sql`.

3 Flask-User Library Overview

The user authentication and credentials management so far have been ad-hoc. For example, we store user credentials in plain text in the database, which is a bad practice. If an attacker obtains the contents of the database they would be able to impersonate any user in the application and possibly beyond (for users that reuse their credentials). Flask-User[3] is a library that provides common authentication functionalities, like login, registration, email confirmation, credential management and recovery. It also provides other features like form security, authorization, and internationalization.

Let's quickly review the main features of Flask-User by modifying the `minisql` application for Flask-SQLAlchemy introduced in Section 2. In the same template `minisql-template`, we refer to the example code in the `user.py` file. All code modifications shown in this section are already applied.

Installation Flask-User is pre-installed in the Docker template, or available via `requirements.txt` for local setups.

Tutorial We highlight the changes introducing password-based authentication:

```
from flask import Flask, render_template_string
```

[3]Full documentation: `https://flask-user.readthedocs.io/en/latest`

```
from flask_sqlalchemy import SQLAlchemy
+ from flask_user import login_required, UserManager, UserMixin, current_user

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'

+ app.config['USER_APP_NAME'] = "Flask-User Mini-App"
+ app.config['USER_ENABLE_EMAIL'] = False
+ app.config['USER_ENABLE_USERNAME'] = True
+ app.config['USER_REQUIRE_RETYPE_PASSWORD'] = False
+ app.config['SECRET_KEY'] = '_5#yfasQ8sansaxec/][#1'

db = SQLAlchemy(app)

- class Person(db.Model):
+ class Person(db.Model,UserMixin):
  id = db.Column(db.Integer, primary_key=True)
  username = db.Column(db.String(100, collation='NOCASE'),
                        nullable=False, unique=True)
  password = db.Column(db.String(255), nullable=False,
                        server_default='')

db.create_all()

+ user_manager = UserManager(app, db, Person)
```

The code above first imports Flask-User classes and then sets some configuration options. To configure a simple password-based authentication (`USER_ENABLE_USERNAME`), we disable the use of emails (`USER_ENABLE_EMAIL`), and do not require users to re-type their passwords during the registration (`USER_REQUIRE_RETYPE_PASSWORD`). The library relies on sessions so we also need to define a secret key. Next, we indicate the data model class representing the user by inheriting the `UserMixin` class. There are some attributes with fixed names that are required from the user class, depending on the features the application needs from Flask-User. Finally, we instantiate the `UserManager` object.

The rest of the code can be replaced with the following routes:

```
@app.route('/')
def home_page():
    return render_template_string('''
{% extends "flask_user_layout.html" %}
{% block content %}
  <h2>Home page</h2>
  <p><a href={{ url_for('user.register') }}>Register</a></p>
  <p><a href={{ url_for('user.login') }}>Sign in</a></p>
  <p><a href={{ url_for('home_page') }}>Home page</a></p>
  <p><a href={{ url_for('member_page') }}>Member page</a></p>
  <p><a href={{ url_for('user.logout') }}>Sign out</a></p>
{% endblock %}
''')
```

```
@app.route('/members')
@login_required
def member_page():
    return render_template_string('''
{% extends "flask_user_layout.html" %}
{% block content %}
  <h2>Members page</h2><p>Hello, {{ username }}</p>
  <p><a href={{ url_for('home_page') }}>Home page</a></p>
  <p><a href={{ url_for('user.logout') }}>Sign out</a></p>
{% endblock %}
''', username=current_user.username)
```

Flask-User comes with pre-defined routes for registration (`user.register`), login (`user.login`), and logout (`user.logout`). The other two routes exemplify a simple authorization feature of the library: the `home_page` route is accessible to anyone, while `member_page` is accessible only to logged in users, which is denoted with the `@login_required` decorator. Flask-User also provides the `current_user` object, which is the instance of the user entity representing the current user.

3.1 Implementing Authentication in the Thoughts Application

Task 2: Authentication for `Thoughts`

Retrieve your solution to the previous task to build upon it.

. . . .

Use the `Flask-User` library to add authentication to the `Thoughts` application. You should:

- configure password-based authentication;
- remove the existing `login`, `logout`, and `register` routes;
- simplify the remaining routes (except for the `index` route) by replacing the manual login checks with the `@login_required` decorator.

. . . .

After the corresponding deadline, the solution will be published in Moodle as `thoughts-user`.

4 Task 3: Implementing Security Policies in the Thoughts Application

Task 3: Security Policies

Retrieve your solution to the previous task to build upon it.

. . . .

Implement the following access control policies in the `Thoughts` application:

(SP1) Any user can read the home page.
(SP2) Only authenticated users can create thoughts.
(SP3) Authenticated users can delete their own thoughts.
(SP4) Authenticated users can vote at most three times for the same thought.
(SP5) Authenticated users are not allowed to vote for their own thoughts.

. . . .

After the corresponding deadline, the solution will be published in Moodle as `thoughts-sec-base`.

To implement policy (SP1), make sure that any user (even non-authenticated users) can access the index route. Conversely for policy (SP2), only authenticated users should be able to access the `add_thought` route. Modify the `delete_thought` route to enforce policy (SP3). The remaining two policies can both be violated in the `vote_thought` route. Modify the route to enforce the policies.

Once you complete this exercise, you may wonder whether these access control policies can be implemented more systematically. Optionally, you can explore some existing Flask libraries that support writing constraints:

- Flask-Authorize: `https://flask-authorize.readthedocs.io/en/latest/`
- Flask-Principal: `https://pythonhosted.org/Flask-Principal/`
- Flask-User: `https://flask-user.readthedocs.io/en/latest/`
- Flask-RBAC: `https://flask-rbac.readthedocs.io/en/latest/`
- Flask-Admin: `https://flask-admin.readthedocs.io/en/latest`

- Oso: `https://www.osohq.com/`
- Cedar: `https://www.cedarpolicy.com/en`
- PyCasbin: `https://github.com/casbin/pycasbin`

These libraries are not yet widely adopted. We will discuss some of them in the next tutorial.

Acknowledgments We would like to thank Marko Lisicic for their feedback on the this tutorial.